

Health Risk Prediction Using Machine Learning Across Heterogeneous Patient Populations

course:CS4082–ML

Instructor: Dr. Naila Marir

1 Hams Aljohani
COE. Computer Science (AI Track)
Effat University
S20106833

2 Ghala Alghamdi
COE. Computer Science (AI Track)
Effat University
S23108212

3 Hiba Amanulla
COE. Computer Science (AI Track)
Effat University
S23108227

Abstract—This project focuses on predicting hospital mortality for ICU patients using machine learning. The data used is the eICU Collaborative Research Database (Demo v2.0), which includes records from multiple hospitals. The dataset was chosen because it reflects real-world clinical diversity, making it suitable for studying how models perform across different patient populations.

The project followed a structured pipeline that includes data collection, preprocessing, feature engineering, model training, hyperparameter tuning, and final evaluation. Several machine learning models were trained and compared, including Logistic Regression, Decision Tree, Random Forest, SVM, KNN, Naive Bayes, Gradient Boosting, and XGBoost. To deal with class imbalance, evaluation focused on AUROC, F1-score, and recall rather than accuracy alone.

I. PROJECT WORKFLOW PIPELINE

1	Problem Definition	Binary classification: Predict hospital mortality (0=Alive, 1=Expired)
2	Data Collection	eICU Demo v2.0: patient.csv, apachePatientResult.csv, apachePredVar.csv, apacheApsVar.csv
3	Preprocessing & EDA	Target creation, missing value handling (>70% drop), leakage removal, data merging & cleaning
4	Feature Engineering	Created is_senior (age≥65), bun_log (log1p transform), PCA exploration, SelectKBest
5	Model Training	8 baseline classifiers: LR, DT, RF, SVM, KNN, NB, GB, XGBoost — with 5-fold CV & adversarial testing
6	Hyperparameter Tuning	GridSearchCV / RandomizedSearchCV for top 3 models (LR, XGBoost, GB)
7	Ensemble & Final Eval	Soft-voting ensemble (LR + XGBoost) evaluated on held-out test set
8	Interpretation & Limitations	Honest analysis of class imbalance, label noise sensitivity, and generalization limits

Fig. 1. The following diagram shows the full pipeline followed in this project, from problem definition through to final model evaluation and interpretation.

II. INTRODUCTION AND MOTIVATION

Predicting patient outcomes in intensive care units (ICU) is one of the most important and challenging tasks in clinical medicine. Doctors and nurses working in ICUs need to make fast and reliable decisions, and having tools that can help identify high-risk patients early can significantly improve care and save lives. Machine learning offers a practical way to build such tools by learning patterns from large amounts of patient data.

The motivation for this project comes from the growing availability of structured clinical data and the increasing use of AI in healthcare. The eICU Collaborative Research Database provides a rich source of anonymized ICU patient records from multiple hospitals across the United States. This multi-center nature of the dataset makes it especially interesting because it introduces heterogeneity, meaning that patient populations, data distributions, and clinical practices may vary between hospitals. This makes it a realistic and challenging setting for machine learning.

The goal of this project is to build models that can predict whether an ICU patient will survive or expire during their hospital stay, based on clinical and demographic information available at or shortly after admission. Beyond just building a model that works on a single dataset, the project also investigates how robust these models are when facing realistic challenges such as class imbalance, noisy labels, and heterogeneous data. The project follows a structured machine learning pipeline, progressing from data preprocessing and feature engineering to model training and evaluation. The final outcome is a voting ensemble model that combines multiple classifiers to achieve more balanced and reliable predictive performance. The project aims to address the research question: Can machine learning models predict hospital mortality using ICU patient data, and how well do they generalize across heterogeneous patient populations? .

III. RELATED WORK

A. Dataset Characteristics and Early Benchmarking Studies

The eICU Collaborative Research Database provides a large multi-center dataset containing patient demographics, vital signs, laboratory results, and clinical outcomes. Although the dataset paper itself does not propose specific machine learning models or report evaluation metrics, it outlines essential preprocessing steps such as data cleaning, de-identification, and structured formatting of clinical variables. Its multi-hospital nature makes it particularly suitable for studying heterogeneous patient populations. Building on this dataset, Sheikhalishahi et al. [1] applied multiple machine learning models, including logistic regression, artificial neural networks (ANN), and Bidirectional Long Short-Term Memory (BiLSTM) networks, to predict mortality and length of stay. Their preprocessing involved separating numerical and categorical variables, applying one-hot encoding and entity embeddings, and using 5-fold cross-validation. The study evaluated performance using AUROC, AUPRC, sensitivity, specificity, mean absolute error (MAE), and R^2 , demonstrating that BiLSTM models outperformed traditional methods due to their ability to capture temporal dependencies. In addition, a comprehensive review of machine learning in healthcare [3] examined commonly used models such as logistic regression, random forest, and deep learning architectures. The review highlighted preprocessing techniques including handling missing values, feature normalization, and feature selection. Evaluation metrics across studies included accuracy and AUROC, confirming the effectiveness of machine learning approaches in healthcare prediction tasks.

B. Explainable and Ensemble Machine Learning Models

Recent research has focused on explainable and interpretable machine learning models, which are essential in clinical settings. An explainable ICU mortality prediction study [2] utilized models such as logistic regression, decision trees, and gradient boosting. The preprocessing involved feature selection using the Cox proportional hazards model to reduce dimensionality while maintaining predictive performance. The primary evaluation metric used was AUC-PR, demonstrating that feature selection enhances interpretability without significantly reducing accuracy. Further advancements are observed in explainable machine learning approaches using XGBoost models [9]. In this work, preprocessing involved feature engineering from time-series data by summarizing variables using statistical measures such as mean and variability over time windows. The model was evaluated using AUROC, average precision, and sensitivity, showing that feature engineering improves both interpretability and predictive performance. Similarly, another study [8] applied random forest, gradient boosting, and logistic regression models for ICU mortality prediction. The preprocessing included handling missing values and performing feature selection, while evaluation metrics included accuracy, AUROC, and F1-score. The results indicated that ensemble models outperform simpler models. In

the context of septic ICU patients, ensemble learning methods such as random forest, extra trees, and support vector machines (SVM) were applied [10]. The preprocessing involved addressing class imbalance using SMOTE and performing hyperparameter tuning through grid search. The models were evaluated using accuracy, AUROC, sensitivity, specificity, and F1-score, demonstrating that ensemble approaches provide strong and stable predictive performance.

C. Deep Learning and Advanced Clinical Prediction Models

Deep learning models have shown strong capability in capturing complex patterns in ICU data. A study using convolutional neural networks (CNNs) [6] applied preprocessing techniques such as time-series feature extraction and handling class imbalance through sampling methods. The model was evaluated using accuracy, root mean square error (RMSE), and R^2 , achieving high predictive performance in mortality and length-of-stay prediction tasks. More advanced work [7] explored hybrid and ensemble machine learning approaches, combining deep learning and traditional models. The preprocessing included data normalization and feature engineering, while evaluation metrics included AUROC, precision, and recall. The results demonstrated that combining multiple models enhances predictive accuracy and robustness.

D. Dataset Shift, Generalization, and Real-World Challenges

A key challenge in healthcare machine learning is dataset shift, where differences between training and deployment data reduce model performance. Subbaswamy and Saria [4] analyzed this issue from a theoretical perspective, emphasizing that models must be robust to distributional changes. This study did not focus on specific models, preprocessing, or metrics but highlighted a fundamental limitation in real-world applications. Generalization across hospitals has also been studied extensively. Zech et al. [5] applied deep learning models and used preprocessing techniques such as data normalization and hospital-based data splitting. Model performance was evaluated using AUROC, and the study showed that models trained on one hospital often perform poorly on others, highlighting generalization challenges in heterogeneous environments. Temporal dataset shift has also been investigated in recent work [11], where random forest and gradient boosting models were used. The preprocessing involved temporal grouping and feature transformation, while evaluation was conducted using AUROC. The results demonstrated that model performance degrades over time due to changes in data distribution, emphasizing the need for robust and adaptive models.

IV. DATA DESCRIPTION AND PREPROCESSING

In this stage, the datasets are prepared for machine learning. This includes creating the target variable, selecting relevant features, merging multiple tables, and handling missing values. The goal is to transform the raw data into a clean and structured format that can be used for model training.

A. Dataset

The dataset used in this project is [12] the eICU Collaborative Research Database (Demo Version 2.0). It contains anonymized ICU patient records collected from multiple hospitals across the United States. Because records come from many different institutions, the dataset reflects real-world clinical diversity and is well-suited for studying how machine learning models perform across heterogeneous patient populations.

Instead of using all available files, four tables were carefully selected to focus on the most clinically relevant features for mortality prediction:

- **patient.csv** Contains patient demographics and hospital discharge status. This table is the source of the target variable.
- **apachePatientResult.csv** Contains APACHE severity scores and outcome-related information collected during the ICU stay.
- **apachePredVar.csv** Includes predicted clinical variables derived from the APACHE scoring system.
- **apacheApsVar.csv** Contains physiological measurements recorded at ICU admission, such as vital signs and laboratory results.

All four tables share the common identifier `patientunitstayid`, which was later used to merge them into a single unified dataset. From the patient table, the column `hospitaldischargestatus` was identified as the most suitable target variable, as it directly indicates whether a patient survived or expired during their hospital stay.

B. Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed before any preprocessing to better understand the structure, distribution, and potential quality issues in the data. This step helps identify important patterns and guides preprocessing decisions.

1) *Class Distribution* : The target variable (target mortality) was first derived from the `hospitaldischargestatus` column in the patient table, where: 0 = survived and 1 = expired. The class distribution was then examined to understand the balance between the two outcome classes.

The result showed a clear class imbalance: approximately 91.6 percent of patients survived and only 8.4 percent had expired. This imbalance is an important finding because it means accuracy alone will be a misleading metric. A model that always predicts survival would still achieve around 91.6 percent accuracy without learning anything useful. For this reason, evaluation in later stages focused primarily on precision, recall, F1-score, and AUROC.

2) *Missing Values* : Missing values were analyzed across all four tables to understand the extent of incomplete data before merging. Both the count and percentage of missing values were calculated for each column.

The findings showed that the level of missingness varied significantly across tables. The patient dataset had moderate missing values in some features. The APACHE predicted variables table contained very high missing percentages for certain

columns. In contrast, the APACHE APS variables dataset contained no missing values at all. These differences indicated that different preprocessing strategies would be needed for different tables, including dropping highly incomplete features and applying imputation for the rest.

3) *Numerical Features* : Histograms were used to examine the distribution of key numerical variables. Patient age appeared relatively spread out across a wide range. The acute physiology score and APACHE score were both right-skewed, with most values concentrated in lower ranges, indicating that extreme severity cases exist but are less common.

Understanding these distributions is important for later preprocessing steps such as scaling and transformation, since skewed features can affect model performance if not handled appropriately.

4) *Outlier Detection* : Boxplots were used to detect outliers in numerical variables. Several features were found to contain extreme values. However, since this is ICU data, extreme measurements often reflect real and critical patient conditions rather than data errors. For example, very high APACHE scores may correspond to patients with severe physiological abnormalities. For this reason, outliers were retained to preserve important clinical information and avoid removing potentially meaningful signal.

5) *Feature Correlation Heatmap* : A correlation heatmap was generated to examine relationships between the key numerical features: Acute Physiology Score, APACHE Score, Predicted ICU Length of Stay, and Predicted Hospital Length of Stay. The heatmap showed that the Acute Physiology Score and APACHE Score were highly correlated, suggesting that they capture similar information about patient severity. The predicted length-of-stay variables showed moderate correlations with the severity scores. These relationships helped identify potential redundancy in the feature space and guided feature selection decisions in later stages.

C. Data Integration

In this step, the four separate tables were merged into a single unified dataset. The patient table was used as the base, and clinical information from the three APACHE-related tables was joined onto it using `patientunitstayid` as the common key.

A critical challenge was that the APACHE tables contain multiple rows per patient because measurements are recorded multiple times during the ICU stay. Directly merging without handling this would produce duplicate patient rows, which would inflate the dataset and create invalid records. To solve this, each APACHE table was first aggregated so that each patient was represented by exactly one row before merging.

1) *Aggregate APACHE Tables* : For the two numerical APACHE tables (`apachePredVar` and `apacheApsVar`), aggregation was performed by grouping rows by `patientunitstayid` and computing the mean for all numeric columns. The mean was chosen because it provides a simple and meaningful summary of patient condition across multiple measurements.

For the `apachePatientResult` table, which contains outcome-related and categorical information, the first available record

per patient was retained rather than computing a mean, since averaging categorical or outcome-related values is not meaningful.

2) *Merge Datasets* : After aggregation, the datasets were merged step by step. The patient table was first merged with the aggregated apachePatientResult table, then with the aggregated apachePredVar table, and finally with the aggregated apacheApsVar table. A left join strategy was applied at each step to preserve all patient records from the patient table, even when no matching rows existed in the APACHE tables.

Note: An initial merge was attempted using the original (non-aggregated) tables, but it resulted in duplicate patient records. This was corrected by first aggregating the APACHE tables before merging.

3) *Handle Duplicate Columns*: After merging, some columns appeared twice with the suffixes '-x' and '-y' because they existed in more than one table. These duplicate columns represented the same variables and therefore contained redundant information. The '-y' versions were removed, and the '-x' suffix was stripped from the remaining column names to restore clean names.

4) *Validate Merge* : A final validation step confirmed that the merged dataset contained no duplicate patientunitstayid values, meaning each patient was correctly represented by exactly one row. After this correction, the merged dataset contained 2520 rows, creating a valid one-patient-per-row structure ready for preprocessing.

D. Data Cleaning

In this step, the merged dataset was cleaned to prepare it for machine learning. This included assessing missing values, removing columns with excessive missing data, removing features that would cause data leakage, and inspecting what remained before applying transformations.

1) *Missing Values Assessment* : Missing values across all columns in the merged dataset were counted and expressed as a percentage of total rows. The results were sorted from highest to lowest missingness to identify which features had the most incomplete data. This helped determine which columns should be dropped and which should be retained for imputation later.

2) *Drop High-Missing Columns* : Columns with more than 70 percent missing values were removed from the dataset. This threshold was chosen deliberately to be relatively high so that important clinical features with moderate missingness were not discarded unnecessarily. Healthcare datasets commonly have incomplete records due to the variability of clinical measurements, so being conservative about dropping helps retain potentially valuable variables. Remaining missing values were not imputed at this stage but were preserved for later handling after the train-test split, to avoid data leakage.

3) *Remove Leakage Features* : Data leakage occurs when the model is given access to information that would not be available at the time of prediction. This is a serious problem in healthcare modeling because it leads to unrealistically high performance that does not transfer to real-world use. Multiple

sequential passes were used to systematically identify and remove all leakage-related columns.

The following categories of features were removed:

- **Direct target leakage:** hospitaldischargestatus, actualhospitalmortality, actualcumortality, predictedhospitalmortality, predictedicumortality, diedinhospital.
- **Discharge and post-outcome information:** hospitaldischargelocation, unitdischargestatus, unitdischargelocation, dischargelocation, hospitaldischargeoffset, hospitaldischargetime24, unitdischargeoffset, unitdischargetime24 .
- **Actual outcome durations recorded after admission:** actualiculos, actualhospitallos, unabridgedunitlos, unabridgedhosplos, actualventdays, unabridgedactualventdays.
- **Model-generated predicted outcome summaries:** predictediculos, predictedhospitallos, predventdays.

Infinite values (inf and -inf) were also replaced with NaN at this point to avoid numerical errors in downstream processing. Multiple additional keyword-based checks were performed to catch any remaining suspicious columns related to discharge, mortality, or length of stay, with the exception of the target variable itself (target-mortality) which was kept.

4) *Inspect Remaining Missing Values* : After removing columns with excessive missingness and leakage columns, the remaining missing values were inspected. At this point, no imputation was applied. Final imputation was deferred until after the train-test split to ensure that imputation statistics were learned only from the training data, which prevents any information from the test set leaking into the preprocessing.

5) *Final Dataset Overview* : After all cleaning steps, the final dataset structure was reviewed. The column data types were checked, the total number of remaining missing values was confirmed, and a preview of the first rows was displayed. The cleaned dataset was then confirmed as ready for feature engineering and model training.

E. Data Transformation

In this step, a preprocessing pipeline was defined to handle all necessary transformations: missing value imputation, numerical scaling, and categorical encoding. These transformations were not applied directly to the data at this stage. Instead, they were packaged inside scikit-learn pipelines that would be applied during model training. This is essential to prevent data leakage — if transformations were applied before splitting, statistics from the test set could influence the training process.

The features were first separated into two groups: numerical features (int64 and float64 columns) and categorical features (object columns). A separate pipeline was then defined for each group:

- **Numerical pipeline:** SimpleImputer with median strategy (to handle missing values robustly against outliers), followed by StandardScaler (to normalize the feature scale so that no single variable dominates the model due to its range).

- **Categorical pipeline:** SimpleImputer with most-frequent strategy (to fill missing values with the most common category), followed by OneHotEncoder with handle-unknown='ignore' (to convert categorical values into binary columns while safely handling unseen categories during testing).

These two pipelines were combined into a single ColumnTransformer called preprocessor, which applied each pipeline to the correct columns. Any remaining infinite values in the numerical features were converted to NaN before the pipeline was finalized, to ensure no invalid values would cause errors during model training.

V. FEATURE ENGINEERING

In this stage, relevant features were selected and refined to improve model performance. This includes creating new features from existing variables and applying transformations to improve data quality and representation. The goal is to create a meaningful set of features that accurately capture the underlying patterns in the data.

A. Feature Creation

Two new features were created from clinically meaningful existing variables to improve the representation of patient risk:

- **is-senior:** A binary feature indicating whether the patient is 65 years or older (1) or not (0). Age is a well-known clinical risk factor for ICU mortality, and this threshold captures an important clinical boundary. The age column was first converted to a numeric format before this feature was created, to handle any non-numeric entries that may have been present in the original data .
- **bun-log:** A log-transformed version of blood urea nitrogen (BUN), a laboratory marker related to kidney function and overall patient severity. BUN values tend to be right-skewed, meaning most values are relatively small while a few are very large. Applying a log1p transformation reduces this skewness and makes the feature distribution more symmetric, which benefits many machine learning algorithms. Invalid values (zeros and infinities) were replaced with NaN before the transformation to prevent numerical errors.

After creating these features, the feature matrix X and target vector y were rebuilt, and the preprocessing pipeline was updated to include the new columns. Any remaining infinite values introduced during feature engineering were replaced with NaN before finalizing.

B. Feature Transformation

Feature transformation was applied to improve data quality and model readiness:

- **Age was converted to numeric format using pd.to-numeric with errors='coerce', which replaces non-numeric entries with NaN, before creating the is-senior feature.**

- **BUN was converted to numeric format, had zero and infinite values replaced with NaN, and was then transformed using np.log1p after clipping to a minimum of zero.**
- **After all feature engineering steps, remaining infinite values across the dataset were replaced with NaN using clean-df.replace([np.inf, -np.inf], np.nan) to ensure the dataset was numerically valid.**

Scaling of numerical variables and encoding of categorical variables were not applied directly here. These were handled through the preprocessing pipeline during model training to prevent leakage.

C. Initial Feature Filtering and Domain Selection

An initial subset of clinically meaningful variables was manually reviewed based on domain knowledge. This included demographic features (age, gender, ethnicity), hospital-related context (hospitalid), admission characteristics (unittype, unitadmitsource, unitstaytype), and engineered features (is-senior, bun-log). The patientunitstayid identifier and the target variable were also included in this review.

This step was used to demonstrate domain-based feature filtering logic and inspect which clinically relevant variables were available. However, the final machine learning models were trained using the broader cleaned dataset (clean-df) after all leakage removal and feature engineering, rather than just this smaller manually selected subset. The broader feature set was used to avoid unnecessarily discarding predictive information.

D. Dimensionality Reduction

Principal Component Analysis (PCA) was applied to the numerical portion of the cleaned dataset as an exploratory step. Only numeric columns were selected, infinite values were replaced with NaN, and remaining NaN values were filled with zero before applying PCA. PCA was configured to extract 2 components, and the explained variance ratio was inspected to understand how much variance could be captured in a reduced number of dimensions.

PCA was used for analysis and dimensionality inspection only. It was not included in the final training pipeline. The final models were trained on the full cleaned and preprocessed feature space rather than on PCA-reduced components, because reducing dimensions could discard predictive information that is useful for classification.

VI. METHODOLOGY

In this stage, multiple machine learning models are trained to predict patient mortality. Different classification algorithms are applied to the prepared dataset to establish baseline performance. The goal is to evaluate how well each model learns from the data and to identify the most effective approaches for further improvement.

A. Algorithm Selection

Eight classification algorithms were selected to provide a broad comparison across different model families, including linear, tree-based, probabilistic, distance-based, and boosting approaches. The selected algorithms and their configurations were:

- **Logistic Regression:** $C=1$, solver=liblinear, class-weight=balanced, max-iter=3000, random-state=42. The class-weight=balanced setting adjusts class weights inversely proportional to class frequencies, which helps the model pay more attention to the minority mortality class.
- **Decision Tree:** random-state=42, class-weight=balanced.
- **Random Forest:** random-state=42, class-weight=balanced.
- **Support Vector Machine (SVM):** probability=True, class-weight=balanced, $C=1$, kernel=rbf, random-state=42.
- **K-Nearest Neighbors (KNN):** default settings.
- **Naive Bayes:** GaussianNB with default settings.
- **Gradient Boosting:** random-state=42 with default hyperparameters.
- **XGBoost:** random-state=42, eval-metric=logloss, scale-pos-weight=($y==0$).sum()/($y==1$).sum(), n-estimators=200, max-depth=3, learning-rate=0.05. The scale-pos-weight parameter was computed from the actual class distribution to handle imbalance by upweighting the minority class during training.

B. Data Splitting Strategy

The dataset was divided into three parts using a two-step stratified splitting strategy. Stratification was applied at each step to preserve the original class distribution (approximately 91.6 percent survived, 8.4 percent expired) across all subsets.

1) *First Split — Train / Test* : The full dataset was split into a training set (80 percent) and a held-out test set (20 percent) using train-test-split with test-size=0.20, random-state=42, and stratify=y. The test set was set aside and not used until the final evaluation stage.

2) *Second Split — Train / Validation* : The training set was further split into a final training set and a validation set using train-test-split with test-size=0.20, random-state=42, and stratify=y-train. The final training set was used for model learning, and the validation set was used to compare model performance and support hyperparameter tuning decisions. This three-way split structure ensured that the test set remained completely unseen during all model development and tuning.

3) *Adversarial Condition Injection* : To simulate real-world data quality challenges, adversarial conditions were introduced into the final training set by randomly flipping 10 percent of the training labels. Label noise of this kind represents scenarios where clinical records are mislabeled, which is a realistic problem in healthcare data collection.

The noise was injected as follows: np.random.seed(42) was set for reproducibility, then noise-ratio=0.10 and n-noisy=int(0.10 * len(y-train-final-noisy)) were computed. A

total of 161 training samples had their labels flipped (0 became 1 and 1 became 0). This noisy training set (y-train-final-noisy) was used later to retrain models and compare adversarial performance against the clean baseline.

Other adversarial techniques such as SMOTE, class weighting, or feature distribution shifts (concept drift) were considered, but label noise injection was selected because it directly simulates real-world mislabeling and provides a clear and interpretable evaluation of model robustness.

C. Baseline Model Training Evaluation

All eight models were trained on the final clean training set and evaluated on the validation set. Each model was wrapped in a scikit-learn Pipeline that applied the preprocessing pipeline first (imputation, scaling, encoding) and then the classifier. This ensures that preprocessing was consistently applied within cross-validation and that no information from the validation set leaked into the training process.

Because the dataset is imbalanced and mortality cases are the minority class, model comparison focused primarily on F1-score, recall, and AUC-ROC rather than accuracy alone. In addition, model-specific decision thresholds were used for selected classifiers to improve minority-class detection:

- **Logistic Regression:** threshold = 0.30
- **XGBoost:** threshold = 0.40.
- **Random Forest:** threshold = 0.35.
- **Gradient Boosting:** threshold = 0.35
- **All other models:** default threshold = 0.50.

1) *Model Training* : All eight baseline pipelines were trained using pipeline.fit(X-train-final, y-train-final) and then evaluated on the validation set using the evaluate-model function, which returned accuracy, precision, recall, F1-score, and AUC-ROC.

2) *Baseline Evaluation Results* : The baseline results showed that XGBoost achieved the strongest overall validation performance, with the highest F1-score of 0.288 and the highest AUC-ROC of 0.790. Logistic Regression provided the next strongest baseline and remained useful as an interpretable comparison model. Other models such as Decision Tree, Naive Bayes, Random Forest, SVM, and KNN showed weaker minority-class performance, especially in recall or F1-score.

Based on these results, XGBoost, Logistic Regression, and Gradient Boosting were identified as the strongest candidate models for further improvement through hyperparameter tuning.

3) *Confusion Matrices* : Confusion matrices were generated for each baseline model on the validation set using sns.heatmap with the Blues colormap. Each matrix provided a detailed breakdown of true positives, false positives, true negatives, and false negatives, helping to understand where each model succeeded and where it struggled, particularly for the minority class.

4) *ROC Curves* : ROC curves were plotted for all eight models on the validation set using the plot-roc-curves function, which computed the false positive rate and true positive rate at every threshold using roc-curve. The AUC-ROC score was

displayed in the legend for each model. These curves gave a visual comparison of how well each model distinguished between the two classes across all decision thresholds.

5) *5-Fold Cross-Validation* : 5-fold stratified cross-validation was performed on all eight models using StratifiedKFold with n-splits=5, shuffle=True, and random-state=42. Cross-validation was performed on the full training set (X-train) using F1-score as the scoring metric, since it accounts for both precision and recall which are more informative than accuracy for imbalanced datasets.

The results confirmed that Logistic Regression and XGBoost were the most consistent models across folds, while weaker models such as KNN and Random Forest struggled with minority-class prediction. The mean F1 and standard deviation were recorded for each model, providing a more reliable estimate of expected performance than single-split validation alone.

6) *Adversarial Comparison* : The same eight baseline models were retrained using the noisy training labels (y-train-final-noisy) that had 161 flipped labels. Their performance on the clean validation set was then evaluated and compared against the clean baseline results. This step directly measured how much each model's performance degraded when trained on mislabeled data.

The adversarial models generally showed reduced precision, F1-score, and AUC-ROC compared with the clean baseline. This confirmed that data quality is an important factor in reliable clinical risk prediction. Some models showed higher recall under noise, but this came at the cost of lower precision and higher false alarm rates.

7) *Baseline vs. Adversarial Comparison* : A side-by-side comparison table was generated that showed, for each model, the Baseline F1, Adversarial F1, F1 Drop, Baseline AUC, Adversarial AUC, and AUC Drop. Models were sorted by Baseline F1 to identify which classifiers maintained the most stability under label noise. This comparison highlighted the importance of clean labeling and showed that robustness analysis is necessary when evaluating healthcare models.

D. Hyperparameter Tuning

Based on baseline performance, three candidate models were selected for hyperparameter tuning: Logistic Regression, XGBoost, and Gradient Boosting. These were chosen for their strong baseline results and their suitability for structured clinical classification. Tuning aimed to find configurations that further improved generalization performance as measured by F1-score on the validation set.

1) *Search Spaces* : The following hyperparameter search spaces were defined for each candidate model:

Logistic Regression (GridSearchCV):

- **model-C:** [0.01, 0.1, 1, 10]
- **model-solver:** [liblinear, lbfgs]

XGBoost (RandomizedSearchCV, n-iter=12):

- **model-n-estimators:** [100, 200]
- **model-max-depth:** [2, 3, 4]
- **model-learning-rate:** [0.03, 0.05, 0.1]

- **model-subsample:** [0.7, 0.8, 1.0]
- **model-colsample-bytree:** [0.7, 0.8, 1.0]

Gradient Boosting (RandomizedSearchCV, n-iter=12):

- **model-n-estimators:** [50, 100, 200]
- **model-learning-rate:** [0.03, 0.05, 0.1]
- **model-max-depth:** [1, 2, 3]
- **model-subsample:** [0.8, 1.0]

GridSearchCV was used for Logistic Regression because its search space is small and exhaustive search is feasible. RandomizedSearchCV with n-iter=12 was used for XGBoost and Gradient Boosting because their search spaces are larger and random sampling is more computationally practical. All searches used 5-fold StratifiedKFold cross-validation with scoring='f1' and n-jobs=-1.

2) *Tuning Results* : After tuning, XGBoost remained the strongest model. However, the improvement over the clean baseline was modest, suggesting that the baseline configuration already captured most of the learnable signal in the data. The tuned models were evaluated on the validation set, and their Validation Accuracy, Validation Precision, Validation Recall, Validation F1, and Validation AUC were recorded alongside the Best Params and Best CV F1 from cross-validation.

This suggests that the dataset structure was already captured reasonably well by the baseline configuration, and that later improvements would more likely come from model combination or robustness-focused strategies rather than parameter tuning alone.

3) *Feature Selection* : Feature selection was applied to the best tuned model (XGBoost) using SelectKBest with mutual information as the scoring function and k=50 as the number of features to retain. This created a new pipeline: preprocessor → SelectKBest → XGBoost. The pipeline was fitted on the final training set and evaluated on the validation set.

The feature-selected version did not outperform the best baseline or tuned model in overall F1-score. This suggests that useful predictive information was distributed across a broader set of features, and removing features through SelectKBest discarded some of that signal rather than improving it.

4) *Final Ensemble* : A soft-voting ensemble was constructed using the tuned Logistic Regression and tuned XGBoost models. Soft voting averages the predicted class probabilities from each model and assigns the class with the highest average probability, rather than using hard majority voting. This approach is generally better when models are well-calibrated, as it makes use of the confidence information in the probability outputs.

The VotingClassifier was fitted on X-train-final and y-train-final. The ensemble was then evaluated on the validation set using the evaluate-model function. The results showed that the soft-voting ensemble produced a stronger balance between precision, recall, and overall discrimination than either single model alone. The ensemble was therefore retained as an important final candidate for test-set evaluation.

VII. EXPERIMENTAL RESULTS AND DISCUSSION

In this stage, the strongest models from the training and tuning stage were evaluated on the held-out test set. The test set was used only once to ensure unbiased final performance reporting. Five candidate models were compared: the clean baseline XGBoost, the tuned XGBoost, the adversarially-trained XGBoost, the XGBoost with SelectKBest feature selection, and the soft-voting ensemble.

A. Final Candidate Model Selection

Based on validation performance, both the strongest baseline model and the strongest tuned model were XGBoost. These were selected as the primary individual candidates. The adversarially-trained version of the best tuned model was also included to assess robustness on the test set. The feature-selected pipeline and the voting ensemble were added as additional candidates to evaluate whether a reduced feature space or model combination could improve final performance.

The adversarial version of the best tuned model was re-trained separately by fitting the same XGBoost configuration on the noisy labels (y-train-final-noisy) rather than the clean labels, so that the test-set comparison would directly measure the impact of label noise on a well-tuned model.

B. Final Test Set Results

The five candidate models were each evaluated once on the held-out test set using the evaluate-model function, with the same model-specific decision thresholds that were applied during validation. The results, sorted by F1-score and AUC-ROC, are shown below:

Model	Accuracy	Precision	Recall	F1-Score	AUC-ROC
Voting Ensemble	0.892857	0.380000	0.452381	0.413043	0.800041
Baseline – XGBoost	0.797619	0.227273	0.595238	0.328947	0.799732
Tuned – XGBoost	0.861111	0.274194	0.404762	0.326923	0.780561
XGBoost + SelectKBest	0.815476	0.200000	0.404762	0.267717	0.725675
Adversarial – XGBoost	0.561508	0.125523	0.714286	0.213523	0.678056

C. Discussion of Results

1) *Voting Ensemble — Best Overall Performance* : On the held-out test set, the Voting Ensemble achieved the best overall performance, with the highest F1-score of 0.413043 and the highest AUC-ROC of 0.800041. It also achieved the highest accuracy (0.892857) among all models. In terms of class-specific performance, the ensemble reached a precision of 0.380000 and recall of 0.452381, indicating a balanced ability to correctly identify mortality cases while controlling false positives. Compared to individual models, the ensemble improved the trade-off between precision and recall, leading to the strongest F1-score. This confirms that combining models (Logistic Regression and XGBoost) enhances robustness and overall predictive performance.

2) *XGBoost — Strong Individual Baseline* : The baseline XGBoost model demonstrated strong performance with an AUC-ROC of 0.799732, which is very close to the ensemble model. Notably, it achieved the highest recall (0.595238) among all models, meaning it was the most effective at identifying true mortality cases. However, this came at the cost of low precision (0.227273), indicating a higher number of false positives. The tuned XGBoost model improved accuracy (0.861111) and precision (0.274194) compared to the baseline, but recall decreased to 0.404762, resulting in a slightly lower F1-score (0.326923) and AUC-ROC (0.780561). This suggests that hyperparameter tuning shifted the model toward a more conservative prediction strategy, improving precision but reducing its ability to detect mortality cases.

3) *XGBoost with SelectKBest — Feature Reduction Impact* : The feature selection approach using SelectKBest resulted in lower performance, with an F1-score of 0.267717 and AUC-ROC of 0.725675. Although recall (0.404762) remained comparable to the tuned model, precision dropped to 0.200000, indicating weaker discrimination between classes. This suggests that reducing the feature set removed important predictive information rather than eliminating noise. The results indicate that ICU mortality prediction benefits from a richer feature space, where multiple variables contribute to model performance.

4) *Adversarial XGBoost — Impact of Label Noise* : The adversarial XGBoost model, trained with 10 percent label noise, showed a significant decline in overall performance. While it achieved a high recall (0.714286), its precision dropped to 0.125523, resulting in a low F1-score (0.213523) and reduced AUC-ROC (0.678056). This indicates that the model became highly sensitive but unreliable, predicting many false positives. The results clearly demonstrate that label noise negatively affects model reliability, which is particularly critical in healthcare applications where data quality directly impacts decision-making.

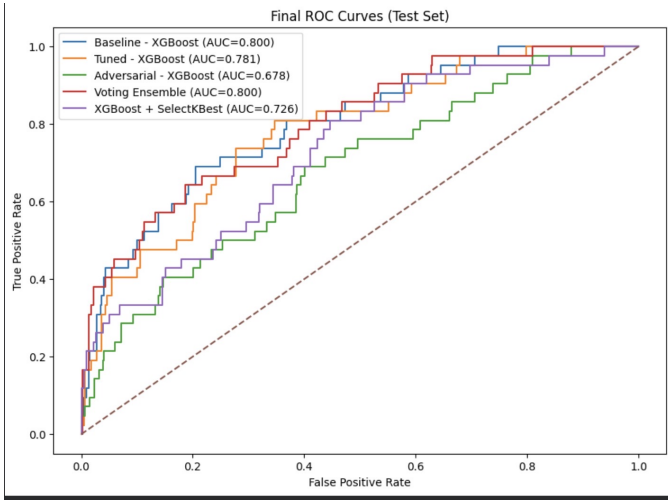
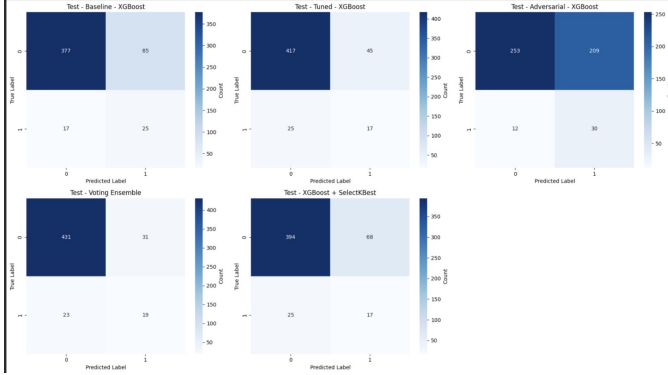
5) *Interpretation of F1-Score* : The best-performing model achieved an F1-score of approximately 0.41, which may appear moderate compared to results on balanced datasets. However, this is a realistic and meaningful outcome for ICU mortality prediction. The dataset is highly imbalanced, with mortality cases representing a small minority of patients. In such scenarios, optimizing both precision and recall simultaneously is challenging. Increasing recall (detecting more true mortality cases) often leads to lower precision (more false positives), and vice versa. The Voting Ensemble successfully achieved the best balance between these competing objectives, improving over all individual models. This indicates that the model provides meaningful predictive performance under realistic clinical conditions, where both sensitivity and reliability are essential.

D. Visualizations

Final confusion matrices and ROC curves were generated for all five candidate models on the test set to provide visual confirmation of the numerical results:

1) *Confusion matrices*: : Generated using `sns.heatmap` with the Blues colormap, annotated with the count of true positives, false positives, true negatives, and false negatives for each model. The matrices made it visually clear which models were better at detecting the minority class versus which ones defaulted to predicting survival for most patients.

2) *ROC curves*: : Plotted for all five candidates on the test set using the `plot-roc-curves` function. The AUC-ROC value was shown in the legend for each model. The ensemble curve consistently ran above the individual model curves at most threshold levels, visually confirming its superior discrimination ability.



E. Final Classification Report

A detailed classification report was generated for the Voting Ensemble (the best-performing model) on the test set using `sklearn.metrics.classification-report` with `zero-division=0`. This report provided class-wise precision, recall, and F1-score for both the survival class (class 0) and the mortality class (class 1), along with the macro and weighted averages. The report confirmed that the ensemble was particularly effective at balancing sensitivity toward the mortality class while maintaining reasonable overall accuracy.

VIII. CONCLUSION AND FUTURE WORK

A. conclusion

This project developed and evaluated machine learning models for predicting hospital mortality in ICU patients using the eICU Collaborative Research Database (Demo Version 2.0). A structured end-to-end pipeline was followed, covering data collection, exploratory data analysis, preprocessing, feature engineering, multi-model baseline training, adversarial robustness testing, hyperparameter tuning, ensemble construction, and final test-set evaluation.

The main findings are:

1) : XGBoost was the strongest individual model, with a baseline AUC-ROC of 0.790 and F1-score of 0.288 on the validation set.

2) : A soft-voting ensemble of tuned Logistic Regression and tuned XGBoost achieved the best final result on the held-out test set, with F1-score = 0.413 and AUC-ROC = 0.800.

3) : Class imbalance (91.6 percent survived vs 8.4percent expired) made accuracy an unreliable metric. Evaluation focused on F1-score, recall, and AUC-ROC throughout.

4) : Adversarial label noise (10 percent , equivalent to 161 flipped training labels) had a measurable negative impact on all models, confirming that data quality is critical for reliable clinical prediction.

5) : Hyperparameter tuning produced only modest improvements over the baseline XGBoost, suggesting that the baseline configuration already captured most learnable patterns in the dataset.

6) : Feature selection using SelectKBest with $k=50$ did not improve performance, indicating that predictive information was spread across the full feature set rather than concentrated in a small subset.

Overall, the project demonstrates that machine learning can provide meaningful support for ICU mortality prediction using structured clinical data. However, the results also highlight the significant challenges of working with imbalanced, noisy, and heterogeneous healthcare data, and show that real-world clinical deployment would require substantially stronger validation, robustness guarantees, and interpretability work.

B. Limitation

1) *Moderate F1-Score and Difficulty of Mortality Prediction* : The final model achieved the strongest overall balance among all tested approaches, but the F1-score remained moderate. This indicates that mortality prediction is still challenging and that the model does not perfectly separate survival and mortality cases. The inherent difficulty of predicting rare adverse outcomes in heterogeneous ICU populations limits what can be achieved with structured tabular data alone.

2) *Class Imbalance* : The dataset contained substantially more survival cases than mortality cases (approximately 91.6 percent vs 8.4 percent). This imbalance influenced model behavior throughout the project, often producing high accuracy without strong minority-class detection. Class weighting was applied in several models, but more advanced strategies such as SMOTE or focal loss were not tested.

3) *Limited Improvement from Hyperparameter Tuning* : Hyperparameter tuning did not produce a large improvement over the baseline XGBoost model. This suggests that the baseline configuration was already reasonably strong for this dataset, or that the dataset itself limits the ceiling of achievable performance for these model families without additional data or features.

4) *Sensitivity to Noisy Labels* : When adversarial label noise was introduced into the training set, performance dropped considerably on the test set. This demonstrates that the model is sensitive to mislabeling and that data quality remains very important in healthcare prediction tasks. Even a relatively small amount of noise (10 percent, or 161 samples) was enough to cause a measurable drop in all metrics.

5) *Feature Selection Trade-Offs* : Applying SelectKBest with $k=50$ did not improve final overall performance, although it remained useful as an experiment for understanding feature importance distributions. This suggests that useful predictive information was distributed across a broader set of features in this dataset.

6) *Single-Dataset Evaluation* : The models were trained and tested on a single demo dataset from the eICU database. Performance may differ substantially when applied to different hospitals, larger patient cohorts, or future ICU admissions not represented in this dataset. No external validation was performed.

7) *Limited Clinical Context* : Only structured variables available in the selected tables were used. Important additional information such as treatment records, time-series monitoring trends, medication history, physician notes, and imaging data was not included. This limits how fully the model can represent the actual complexity of patient health trajectories.

C. Future Work

Based on the limitations identified, several directions could meaningfully improve this work:

- **External Validation:** Validate the models on external hospital datasets or the full eICU database (not just the demo version) to assess cross-site generalization. Evaluating performance across different hospitals would directly test the robustness claims made in this project.
- **Time-Series Modeling:** Incorporate time-series ICU monitoring data using recurrent or temporal models such as LSTM or Transformer architectures, to capture how patient condition evolves over the ICU stay rather than relying only on summary statistics.
- **Advanced Imbalance Handling:** Apply more sophisticated techniques for handling class imbalance, such as SMOTE (Synthetic Minority Over-sampling Technique), focal loss, or cost-sensitive learning, to improve minority-class detection without sacrificing precision.
- **Explainability:** Use SHAP (SHapley Additive exPlanations) or LIME to produce feature-level explanations for individual predictions. This would make model outputs more transparent and trustworthy to clinical staff, which is a requirement for real-world deployment.

- **Threshold Optimization:** Systematically optimize decision thresholds for the final model based on desired precision-recall tradeoffs, using domain-specific cost functions that reflect the relative clinical consequences of false negatives (missed mortality cases) versus false positives (unnecessary alarms).
- **Robustness Improvements:** Explore label noise-robust training approaches, such as learning with noisy labels techniques or confidence-based sample weighting, to reduce the sensitivity to mislabeled training data observed in this project.

REFERENCES

- [1] Sheikhalishahi, S., Balaraman, V., and Osmani, V. (2020). Benchmarking machine learning models on multi-centre eICU critical care dataset. PLOS ONE. <https://journals.plos.org/plosone/article?id=10.1371/journal.pone.0235424>
- [2] Machine learning in healthcare: Review and applications. (2022). Physiological Reviews. <https://journals.physiology.org/doi/10.1152/physrev.00033.2022>
- [3] Explainable machine learning prediction of ICU mortality. (2021). Journal of Biomedical Informatics. <https://www.sciencedirect.com/science/article/pii/S2352914821001593>
- [4] Subbaswamy, A., and Saria, S. (2020). From development to deployment: Dataset shift, causality, and shift-stable models in health AI. Biostatistics, 21(2), 345–361. <https://academic.oup.com/biostatistics/article/21/2/345/5613899>
- [5] Zech, J. R., Badgeley, M. A., Liu, M., Costa, A. B., Titano, J. J., and Oermann, E. K. (2018). Variable generalization performance of a deep learning model across hospitals. PLOS Medicine. <https://journals.plos.org/plosmedicine/article?id=10.1371/journal.pmed.1002683>
- [6] Deep learning approach for ICU mortality and length of stay prediction. (2023). Journal of Biomedical Informatics. <https://www.sciencedirect.com/science/article/pii/S1532046423002472>
- [7] Advanced machine learning for clinical prediction. (2025). npj Digital Medicine. <https://www.nature.com/articles/s41746-025-02114-y>
- [8] Machine learning models for ICU mortality prediction. (2025). PMC. <https://pmc.ncbi.nlm.nih.gov/articles/PMC12975804/>
- [9] Explainable machine learning for ICU mortality prediction. (2025). Scientific Reports. <https://www.nature.com/articles/s41598-025-13299-3>
- [10] Machine learning for mortality prediction in septic ICU patients. (2025). Journal of Clinical Medicine. <https://www.mdpi.com/2077-0383/14/10/3495>
- [11] Temporal dataset shift in ICU data and its impact on model performance. (2025). ScienceDirect. <https://www.sciencedirect.com/science/article/pii/S229196942500287X>
- [12] [12] Johnson, A., Pollard, T., Badawi, O., and Raffa, J. (2019). eICU Collaborative Research Database Demo (version 2.0). PhysioNet. <https://physionet.org/content/eicu-crd-demo/2.0.1/>